



Licenciatura em Engenharia Informática e Multimédia

Desenvolvimento de Aplicações Móveis

Projeto Final

Forge 5e

Aplicação de Gestão de Conteúdo e de Fichas de Personagens de
Dungeons and Dragons

Emiliana Prates Nº Aluno: 51635

Semestre Verão 2025/2026

Índice

Índice.....	1
Índice de Figuras.....	2
Introdução.....	3
Motivação.....	4
Objetivos do Projeto.....	5
Objetivos Específicos.....	5
Objetivos Técnicos.....	6
Arquitetura do Projeto.....	6
Organização Geral.....	6
Camada View.....	7
Camada ViewModel.....	7
Camada Model.....	8
Repositórios.....	8
Comunicação com API externa.....	9
Fluxo de dados.....	9
Vantagens da Arquitetura Utilizada.....	10
Funcionamento da Aplicação.....	11
Ecrã inicial.....	11
Gestão de personagens.....	12
Criação de personagem.....	13
Consulta da ficha de personagem.....	17
Navegador de conteúdo SRD.....	22
Simulador de lançamento de dados.....	23
Navegação e experiência de utilização.....	24
Modelo de Dados.....	24
Entidade CharacterEntity.....	24
Entidades associadas à personagem.....	25
Relação entre entidades.....	26
Acesso aos dados locais.....	26
Integração com a API SRD 5e.....	27
Dados locais vs. dados remotos.....	28
Conclusão.....	28

Índice de Figuras

Figura 1 — Ecrã inicial da aplicação Forge 5e.....	11
Figura 2 — Lista de personagens guardadas.....	12
Figura 3 — Primeira etapa do Character Builder.....	13
Figura 4 — Definição dos atributos da personagem.....	14
Figura 5 — Escolha da raça e background.....	15
Figura 6 — Escolha da classe, proficiências e equipamento inicial.....	16
Figura 8 — Separador Main da ficha de personagem.....	17
Figura 9 — Separador Attacks.....	18
Figura 10 — Separador Spells.....	19
Figura 11 — Separador Equipment.....	20
Figura 12 — Separador Features.....	21
Figura 13 e 14 — Navegador SRD com lista de categorias e Detalhe de um item SRD selecionado.....	22
Figuras 15 e 16 — Ecrã Dice Roller antes do lançamento e Resultado de um lançamento de dados.....	23

Introdução

O presente relatório descreve o desenvolvimento da aplicação **Forge 5e**, uma aplicação Android destinada à gestão de personagens de **Dungeons & Dragons 5.^a edição**. O projeto foi desenvolvido no âmbito do projeto final da unidade curricular, tendo como objetivo aplicar conceitos de desenvolvimento Android, organização de código, construção de interfaces e persistência de dados numa aplicação funcional e temática.

A ideia inicial apresentada ao docente consistia numa aplicação de gestão de personagens de Dungeons & Dragons. Na versão implementada, essa proposta foi expandida para incluir não só a criação e consulta de personagens, mas também funcionalidades complementares associadas à utilização prática de uma ficha de personagem durante uma campanha. A aplicação possui um menu principal com acesso às áreas de personagens, consulta de conteúdo SRD e lançamento de dados, demonstrando uma divisão clara das principais funcionalidades da aplicação.

A aplicação foi desenvolvida em Kotlin para Android, utilizando **Jetpack Compose** para a construção da interface gráfica e navegação entre ecrãs. A estrutura principal da aplicação inclui ecrãs para o menu inicial, lista de personagens, detalhe da personagem, construtor de personagens, navegador de conteúdo SRD (System Reference Document) e simulador de dados.

O Forge 5e utiliza também persistência local através de **Room**, permitindo guardar personagens e respetiva informação associada, como equipamento, magias, características e recursos personalizados. Além disso, a aplicação integra uma API externa de D&D 5e para obter informação sobre classes, raças, magias, equipamento, talentos, backgrounds e outros elementos do sistema de jogo.

Motivação

A motivação para o desenvolvimento do Forge 5e surgiu da necessidade de simplificar a gestão de personagens em sessões de Dungeons & Dragons. Durante uma campanha, os jogadores precisam de consultar frequentemente atributos, pontos de vida, proficiências, equipamento, magias, ataques e características da personagem. Quando esta informação é mantida apenas em papel ou em documentos separados, a sua atualização e consulta pode tornar-se pouco prática.

Assim, a aplicação pretende oferecer uma solução móvel que centraliza a informação essencial de uma personagem num único local. Através do Forge 5e, o utilizador pode criar personagens, consultar a sua ficha, acompanhar pontos de vida, nível, experiência, dados de combate, proficiências, magias, equipamento e características obtidas a partir da raça, classe, subclasse e background. O ecrã de detalhe da personagem está organizado em separadores, incluindo áreas como **Main**, **Attacks**, **Spells**, **Equipment** e **Features**, o que permite uma consulta mais estruturada da informação.

Outro fator de motivação foi a possibilidade de criar uma aplicação com complexidade suficiente para demonstrar vários aspectos importantes do desenvolvimento Android. O projeto não se limita a uma lista simples de dados; inclui armazenamento local, comunicação com uma API externa, navegação entre vários ecrãs, validação de formulários, lógica de cálculo de atributos e pontos de vida, gestão de inventário, lançamento de dados e atualização dinâmica da interface com base no estado da aplicação.

A existência de um **construtor de personagens** também reforça a utilidade da aplicação. Este construtor guia o utilizador por várias etapas, como definição de regras opcionais, escolha de atributos, seleção de raça e background, escolha de classe e subclasse, e seleção opcional de talento inicial. Esta abordagem torna o processo de criação mais organizado e reduz a probabilidade de o utilizador se esquecer de preencher informação importante.

Além disso, a inclusão de um navegador de conteúdo SRD permite que a aplicação funcione também como ferramenta de consulta. O utilizador pode navegar por categorias como classes, raças, magias, talentos, backgrounds e equipamento, consultando informação relevante sem sair da aplicação.

Objetivos do Projeto

O objetivo geral do projeto é desenvolver uma aplicação Android funcional, intuitiva e visualmente adequada ao tema de Dungeons & Dragons, capaz de apoiar a criação, gestão e consulta de personagens da 5.ª edição do jogo.

A aplicação pretende servir como uma ficha digital interativa, permitindo ao utilizador guardar personagens localmente, consultar informação relevante sobre as mesmas e utilizar ferramentas auxiliares durante o jogo, como o lançamento de dados e a consulta de conteúdo SRD.

Objetivos Específicos

Os objetivos específicos do projeto são:

- Desenvolver uma aplicação Android com interface moderna utilizando **Jetpack Compose**.
- Criar um menu principal que permita navegar para as principais funcionalidades: personagens, navegador SRD e simulador de dados.
- Implementar uma lista de personagens guardadas, permitindo consultar detalhes e eliminar personagens existentes.
- Criar um construtor de personagens dividido em etapas, permitindo definir nome, regras opcionais, atributos, raça, background, classe, subclasse, proficiências, equipamento inicial e talento opcional.
- Guardar os dados das personagens de forma persistente através de uma base de dados local Room.
- Representar uma personagem com informação detalhada, incluindo atributos, pontos de vida, dados de morte, exaustão, experiência, proficiências, moedas, magias, equipamento e regras opcionais.
- Permitir a gestão de equipamento, magias, características e recursos associados a cada personagem.
- Integrar uma API externa de Dungeons & Dragons 5e para obter dados sobre classes, raças, magias, equipamento, talentos, backgrounds, proficiências e características.
- Disponibilizar um navegador de conteúdo SRD com categorias e detalhes dos elementos do jogo.
- Implementar um simulador de lançamento de dados que permita escolher o tipo de dado, o número de dados e um modificador ao resultado.
- Criar uma interface organizada e acessível, com separação por ecrãs, separadores e componentes reutilizáveis.

Objetivos Tecnicos

Para além dos objetivos funcionais, o projeto tem também objetivos técnicos relacionados com a qualidade da implementação:

- Aplicar uma arquitetura organizada, separando modelos de dados, acesso à base de dados, repositórios, ViewModels e componentes de interface.
- Utilizar **StateFlow** e ViewModels para gerir o estado da interface de forma reativa.
- Usar o padrão de repositório para separar a lógica de acesso a dados locais e remotos.
- Implementar comunicação HTTP com Retrofit e Kotlinx Serialization, incluindo configuração para lidar com dados desconhecidos da API externa.
- Garantir que a aplicação possui uma complexidade superior a uma aplicação CRUD simples, através da inclusão de cálculos de jogo, filtros de magias por classe e nível, gestão de descansos, ataques, equipamento e características da personagem.

Arquitetura do Projeto

A aplicação **Forge 5e** foi desenvolvida seguindo uma arquitetura baseada no padrão **MVVM** — **Model-View-ViewModel**. Esta arquitetura foi escolhida por permitir uma melhor separação de responsabilidades entre a interface gráfica, a lógica da aplicação e o acesso aos dados. Desta forma, o código torna-se mais organizado, reutilizável e fácil de manter.

No contexto da aplicação, a camada **View** é responsável pela apresentação da interface ao utilizador, a camada **ViewModel** gere o estado da aplicação e a lógica necessária para cada ecrã, enquanto a camada **Model** representa os dados e os mecanismos de acesso a informação local e remota.

Organização Geral

A aplicação está organizada em várias áreas principais. A interface gráfica encontra-se maioritariamente nos pacotes associados a **ui**, incluindo ecrãs e componentes reutilizáveis. Os ViewModels encontram-se no pacote **viewModel**, enquanto os dados, entidades, repositórios, API e modelos de domínio estão organizados dentro do pacote **model**.

Esta divisão permite separar claramente o código relacionado com a apresentação visual do código responsável pela lógica de negócio e persistência de dados. Por exemplo, a navegação principal da aplicação é definida na **MainActivity**, que cria os repositórios necessários e associa cada ecrã ao respetivo ViewModel. A aplicação possui rotas para o menu principal, lista de personagens, detalhe da personagem, construtor de personagens, navegador SRD e simulador de dados.

Camada View

A camada **View** é responsável pela interface gráfica e foi implementada com **Jetpack Compose**. Esta camada inclui os vários ecrãs da aplicação, como o menu principal, a lista de personagens, o detalhe da personagem, o construtor de personagens, o navegador SRD e o simulador de dados.

Nesta arquitetura, os ecrãs não devem conter diretamente a lógica principal da aplicação. Em vez disso, observam o estado fornecido pelos ViewModels e invocam funções desses ViewModels quando o utilizador realiza alguma ação. Um exemplo disso encontra-se no ecrã da lista de personagens, onde a lista é obtida através do `CharacterViewModel` e observada com `collectAsState()`.

O ecrã de detalhe da personagem também segue esta abordagem. Este ecrã recebe o `CharacterDetailViewModel`, observa a personagem atual e organiza a informação em separadores como **Main**, **Attacks**, **Spells**, **Equipment** e **Features**.

Esta organização permite que a interface seja reativa: quando os dados são alterados no ViewModel ou na base de dados, a interface é automaticamente atualizada.

Camada ViewModel

A camada **ViewModel** é responsável por gerir o estado da aplicação e intermediar a comunicação entre a interface e os dados. No Forge 5e existem vários ViewModels, cada um associado a uma área funcional da aplicação, como:

- `CharacterViewModel`
- `CharacterDetailViewModel`
- `CharacterBuilderViewModel`
- `SrdBrowserViewModel`
- `DiceRollerViewModel`

O `CharacterViewModel`, por exemplo, expõe a lista de personagens como um `StateFlow`, convertendo o fluxo de dados recebido do repositório num estado observável pela interface. Também contém ações como adicionar e eliminar personagens.

O `CharacterBuilderViewModel` gere o estado do processo de criação de personagens. Este ViewModel mantém informação como o passo atual do formulário, nome da personagem, atributos, raça, subraça, classe, background, equipamento inicial, proficiências e talentos.

Já o `CharacterDetailViewModel` possui uma responsabilidade mais complexa, uma vez que gere a ficha completa da personagem. Este ViewModel observa a personagem, inventário, magias, recursos e características através de `StateFlow`, além de calcular valores derivados como iniciativa, percepção passiva, classe de armadura e ataques disponíveis.

Os ViewModels utilizam `viewModelScope` para executar operações assíncronas, como chamadas à base de dados ou à API externa, sem bloquear a interface do utilizador. Esta abordagem melhora a fluidez da aplicação e evita que operações demoradas afetem a experiência do utilizador.

Camada Model

A camada **Model** contém a representação dos dados e os mecanismos de acesso aos mesmos. No Forge 5e, esta camada inclui entidades da base de dados, DAOs, repositórios, serviços de API, DTOs e modelos de domínio.

A persistência local é implementada com **Room**. A base de dados da aplicação é definida na classe `AppDatabase`, que inclui entidades como `CharacterEntity`, `CharacterEquipmentEntity`, `CharacterSpellEntity`, `CharacterTrackerEntity` e `CharacterFeatureEntity`.

A entidade principal da aplicação é `CharacterEntity`, que representa uma personagem criada pelo utilizador. Esta entidade guarda informação como nome, classe, raça, nível, atributos, pontos de vida, proficiências, moedas, magias, regras opcionais e talento inicial.

Para além da personagem principal, existem entidades associadas a equipamento, magias, características e recursos personalizados. Estas entidades permitem representar uma ficha de personagem de forma mais completa e estruturada.

O acesso direto à base de dados é feito através do `CharacterDao`, que contém operações para consultar, inserir, atualizar e eliminar personagens, equipamento, magias, recursos e características. Algumas consultas devolvem `Flow`, permitindo que as alterações na base de dados sejam refletidas automaticamente na interface.

Repositórios

Os repositórios funcionam como uma camada intermédia entre os ViewModels e as fontes de dados. Esta abordagem evita que os ViewModels comuniquem diretamente com a base de dados ou com a API, tornando o código mais modular.

O `CharacterRepository` encapsula as operações relacionadas com personagens guardadas localmente. Este repositório disponibiliza métodos para inserir, eliminar e consultar personagens, bem como gerir equipamento, magias, recursos e características associadas a cada personagem.

O `SrdRepository` é responsável por abstrair o acesso à API de Dungeons & Dragons 5e. Este repositório fornece métodos para obter listas e detalhes de classes, raças, magias, equipamento, talentos, backgrounds, subclasses, subraças, proficiências e características.

Esta separação é importante porque permite que os ViewModels trabalhem com uma interface mais simples, sem precisarem de conhecer os detalhes técnicos da base de dados ou da comunicação HTTP.

Comunicação com API externa

Para obter informação do sistema SRD de Dungeons & Dragons 5e, a aplicação utiliza uma API externa. A comunicação é feita através de **Retrofit**, configurado no objeto **NetworkClient**. Este objeto define o URL base da API, configura o cliente HTTP e cria uma instância do serviço **SrdApiService**.

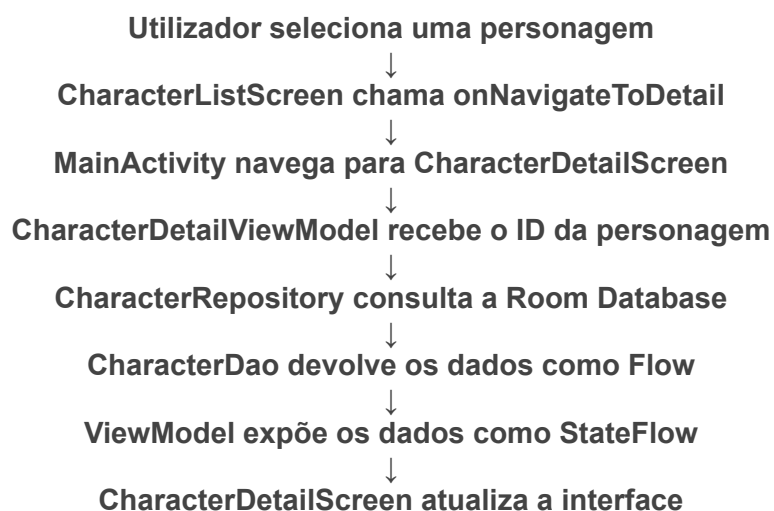
O **SrdApiService** define os endpoints disponíveis através de anotações **@GET**, permitindo obter dados como classes, raças, magias, equipamento, backgrounds, subclasses, subraças, proficiências e características.

Para evitar que a interface dependa diretamente dos modelos recebidos da API, existem funções de mapeamento que convertem DTOs da API em modelos de domínio mais adequados à apresentação na aplicação. Esta separação reduz o acoplamento entre a camada de rede e a interface gráfica.

Fluxo de dados

O fluxo de dados da aplicação segue o padrão típico de MVVM. Quando o utilizador interage com a interface, por exemplo ao criar uma personagem, equipar um item ou consultar uma magia, a View chama uma função do ViewModel. O ViewModel processa essa ação e, quando necessário, comunica com um repositório. O repositório, por sua vez, acede à base de dados local ou à API externa.

Quando os dados são alterados, os **Flow** e **StateFlow** propagam essas mudanças de volta para o ViewModel e para a View. Assim, a interface é atualizada automaticamente com o novo estado da aplicação.



Vantagens da Arquitetura Utilizada

A utilização da arquitetura MVVM trouxe várias vantagens para o desenvolvimento do Forge 5e. Em primeiro lugar, permitiu separar a interface gráfica da lógica da aplicação, tornando os ecrãs mais simples e focados apenas na apresentação dos dados. Em segundo lugar, os ViewModels centralizam o estado e a lógica de cada funcionalidade, facilitando a manutenção e evolução do projeto.

Outra vantagem é a integração natural com o Jetpack Compose, uma vez que a interface pode observar estados expostos pelos ViewModels e atualizar-se automaticamente quando esses estados mudam. Além disso, a utilização de repositórios tornou possível isolar o acesso à base de dados e à API externa, melhorando a organização e reduzindo o acoplamento entre componentes.

Assim, a arquitetura MVVM adotada no Forge 5e contribui diretamente para a qualidade do código, para a escalabilidade da aplicação e para a sua manutenção futura.

Funcionamento da Aplicação

Esta secção apresenta o funcionamento da aplicação **Forge 5e** do ponto de vista do utilizador final. O objetivo é demonstrar o fluxo de utilização da aplicação, os principais ecrãs disponíveis e a forma como as funcionalidades implementadas apoiam a criação, gestão e consulta de personagens de **Dungeons & Dragons 5e**.

Ecrã inicial

Ao iniciar a aplicação, o utilizador é recebido pelo menu principal do **Forge 5e**. Este ecrã apresenta o nome da aplicação e uma breve descrição da sua finalidade: servir como uma ferramenta de gestão de fichas de personagem e consulta de conteúdo relacionado com Dungeons & Dragons 5e.

A partir do menu principal, o utilizador pode aceder às três áreas principais da aplicação: a gestão de personagens, o navegador de conteúdo SRD e o simulador de lançamento de dados. Esta organização torna a navegação simples e direta, permitindo que o utilizador escolha rapidamente a funcionalidade pretendida.

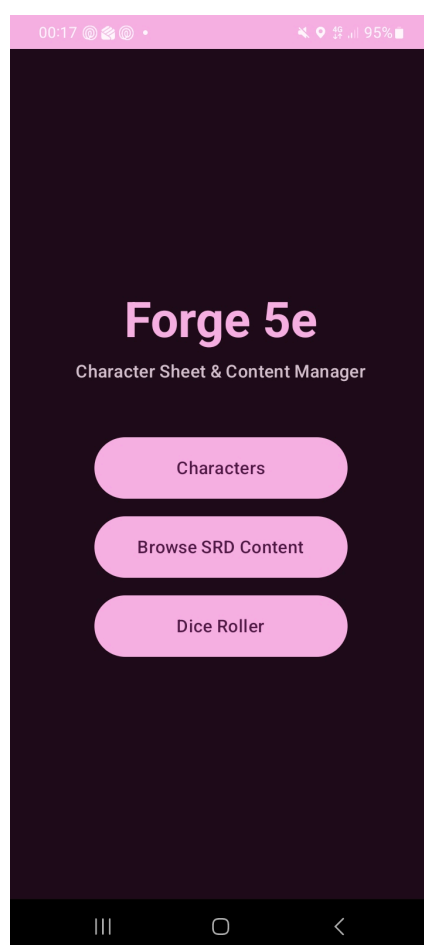


Figura 1 — Ecrã inicial da aplicação Forge 5e

Gestão de personagens

A opção **Characters** permite ao utilizador aceder à lista de personagens guardadas na aplicação. Neste ecrã são apresentados cartões com a informação resumida de cada personagem, incluindo o nome, o nível, a raça e a classe. Esta apresentação permite identificar rapidamente cada personagem existente.

Cada cartão pode ser selecionado para abrir a ficha detalhada da personagem. Além disso, existe um botão de eliminação associado a cada personagem, permitindo remover personagens que já não sejam necessárias. No canto inferior do ecrã existe também um botão de ação para criar uma nova personagem.

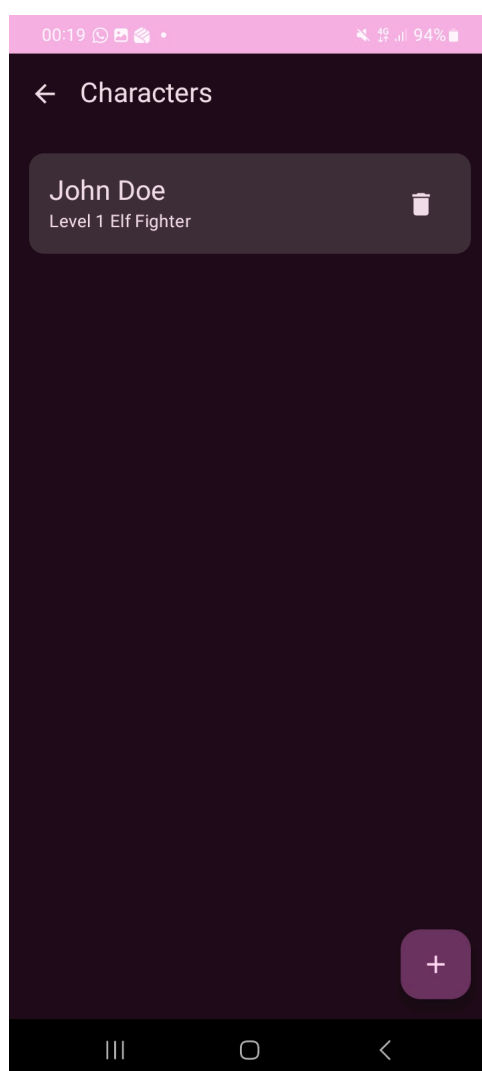


Figura 2 — Lista de personagens guardadas

Criação de personagem

A criação de uma nova personagem é feita através de um **Character Builder**, organizado em várias etapas. Esta abordagem guia o utilizador ao longo do processo de criação, evitando que toda a informação seja apresentada de uma só vez e tornando a experiência mais clara.

Na primeira etapa, o utilizador introduz o nome da personagem e define algumas preferências ou regras opcionais. Entre estas opções encontram-se o tipo de progressão da personagem, por experiência ou por marcos narrativos, a utilização de regras de carga, o método de cálculo dos pontos de vida e a possibilidade de usar talentos.

00:18 95%

← Character Builder - Step 1/4

Basic Information

Character Name

Variant Rules & Preferences

Level-Up Progression
Experience Points (XP) ☐

Use Encumbrance
Calculate weight of items and speed penalties ☒

HP Calculation Mode

☒ Fixed Average

☐ Always Maximum

☐ Standard Roll

☐ Roll (Take average if lower)

Use Feats
Allow selecting feats instead of Ability Score Improvements ☐

Next

Figura 3 — Primeira etapa do Character Builder

Na segunda etapa, o utilizador define os valores base dos atributos da personagem: Força, Destreza, Constituição, Inteligência, Sabedoria e Carisma. Estes valores são introduzidos através de controlos deslizantes, tornando o processo mais visual e simples.

The screenshot shows a mobile application interface for 'Character Builder - Step 2/4'. The title bar is dark blue with a back arrow and the text 'Character Builder - Step 2/4'. Below the title bar, the section is titled 'Ability Scores' with a subtitle 'Manually enter your base ability scores (before racial bonuses)'. There are six rows of sliders for different attributes: STR (Strength), DEX (Dexterity), CON (Constitution), INT (Intelligence), WIS (Wisdom), and CHA (Charisma). Each row has a slider with a vertical line indicating the current value and a numerical value to the right. The values are: STR 15, DEX 12, CON 8, INT 12, WIS 11, and CHA 15. At the bottom of the screen, there are two buttons: 'Back' and 'Next'. The 'Next' button is highlighted in red. The bottom of the screen shows the Android navigation bar with three icons: a square, a circle, and a triangle.

Attribute	Value
STR	15
DEX	12
CON	8
INT	12
WIS	11
CHA	15

Figura 4 — Definição dos atributos da personagem

De seguida, o utilizador escolhe a raça e o background da personagem. Quando uma raça é selecionada, a aplicação apresenta informação adicional, como características raciais e bónus de atributos. Da mesma forma, ao seleccionar um background, são mostradas as proficiências associadas.

The screenshot shows a mobile application interface for character creation. At the top, a pink status bar displays the time 00:19, signal strength, and 94% battery. Below this, a dark purple header bar contains a back arrow and the text 'Character Builder - Step 3/4'. The main title 'Race & Background' is centered. Three dropdown menus are stacked vertically: 'Race' with 'Elf' selected, 'Subrace' with 'High Elf' selected, and 'Background' with 'Acolyte' selected. Below these are two light purple boxes. The first box, titled 'Racial Traits', lists 'Darkvision', 'Fey Ancestry', 'Trance', and 'Keen Senses', followed by 'Ability Bonuses: +2 DEX'. The second box, titled 'Background Proficiencies', lists 'Skill: Insight' and 'Skill: Religion'. At the bottom, a dark purple bar contains a 'Back' button and a pink 'Next' button. The very bottom of the screen shows the standard Android navigation bar with three icons.

Figura 5 — Escolha da raça e background

A etapa seguinte permite escolher a classe e, quando disponível, a subclasse da personagem. Nesta fase são apresentados detalhes importantes da classe, como o dado de vida e os testes de resistência. O utilizador pode também seleccionar proficiências e opções de equipamento inicial, de acordo com as escolhas disponíveis para a classe seleccionada.

00:33 4G 93%

← Character Builder - Step 4/4

Class & Subclass

Class
Druid

Subclass
Select Subclass

Class Details
Hit Die: d8
Saving Throws: INT, WIS

Choose two from Arcana, Animal Handling, Insight, Medicine, Nature, Perception, Religion, and Survival (2/2)

☐ Arcana

☒ Animal Handling

☐ Insight

☐ Medicine

Back Finish

Figura 6 — Escolha da classe, proficiências e equipamento inicial

Após completar as etapas necessárias, o botão **Finish** conclui a criação da personagem e guarda-a localmente na aplicação.

Consulta da ficha de personagem

Depois de criada, a personagem pode ser aberta a partir da lista de personagens. A ficha detalhada está organizada por separadores, permitindo consultar diferentes áreas da personagem sem sobrecarregar o ecrã. Os separadores disponíveis são **Main**, **Attacks**, **Spells**, **Equipment** e **Features**.

No separador **Main**, o utilizador encontra a informação principal da personagem. Aqui são apresentados dados como nível, experiência, pontos de vida, estatísticas de combate, dados de vida, testes contra morte, exaustão, bónus de proficiência, percepção passiva, atributos, testes de resistência, perícias e proficiências. Este ecrã funciona como a área central da ficha de personagem.

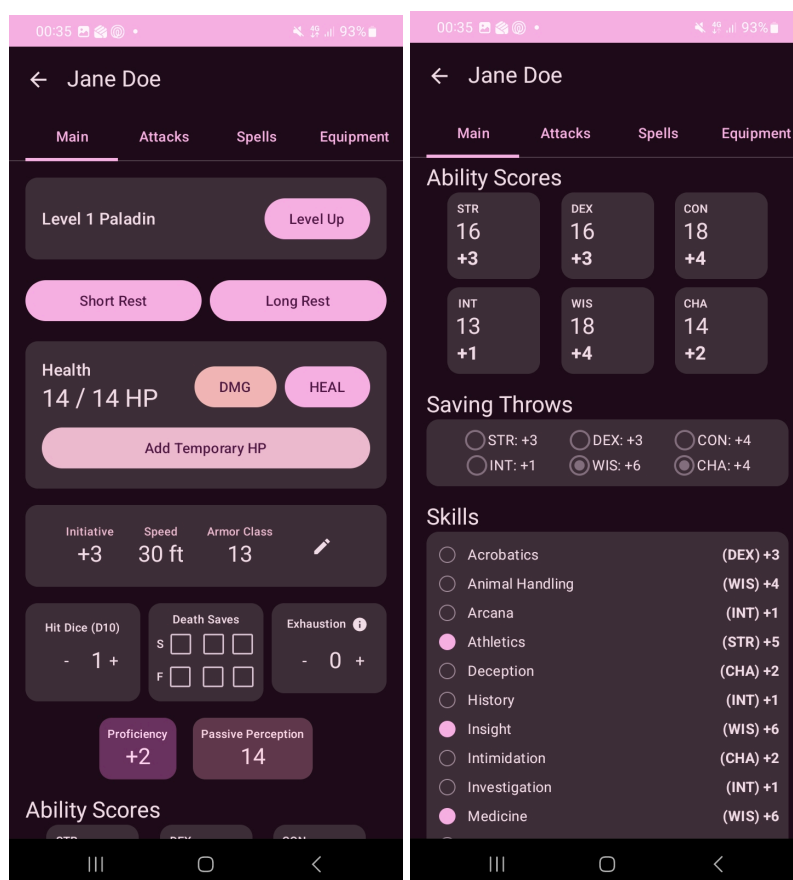


Figura 8 — Separador Main da ficha de personagem

Ainda neste separador, o utilizador pode realizar ações relacionadas com descansos curtos e longos. Estas funcionalidades são úteis durante uma sessão de jogo, pois permitem atualizar recursos da personagem de forma mais rápida.

No separador **Attacks**, são apresentados os ataques disponíveis da personagem. Caso a personagem não tenha ataques disponíveis, a aplicação informa o utilizador de que deve equipar armas ou adicionar magias. Esta área permite consultar rapidamente as opções ofensivas da personagem durante o jogo.

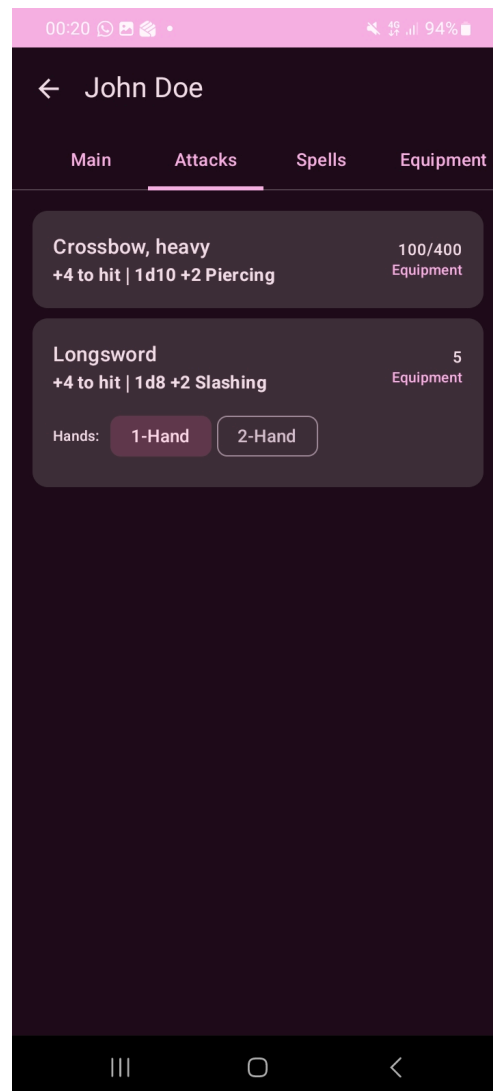


Figura 9 — Separador Attacks

O separador **Spells** é dedicado à gestão de magias. Neste ecrã, o utilizador pode consultar estatísticas de conjuração, ver magias conhecidas ou preparadas e adicionar novas magias a partir do conteúdo SRD disponível. A aplicação distingue entre personagens que preparam magias e personagens que apenas possuem uma lista fixa de magias conhecidas.

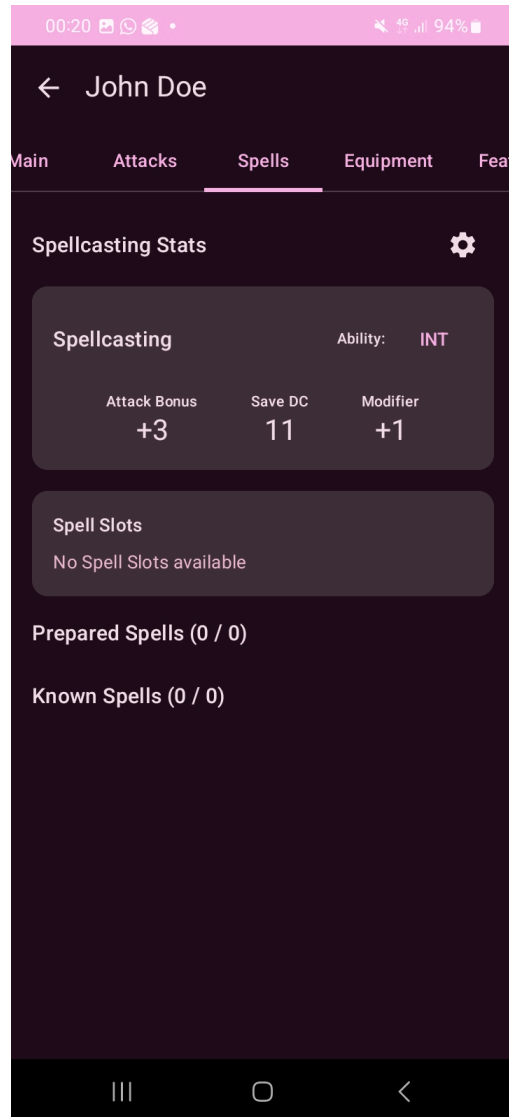


Figura 10 — Separador Spells

No separador **Equipment**, o utilizador pode consultar e gerir o inventário da personagem. São apresentados elementos como carga transportada, capacidade de transporte, moedas e lista de itens. O utilizador pode adicionar equipamento a partir do SRD, marcar itens como equipados, consultar informação adicional sobre cada item e remover itens do inventário.

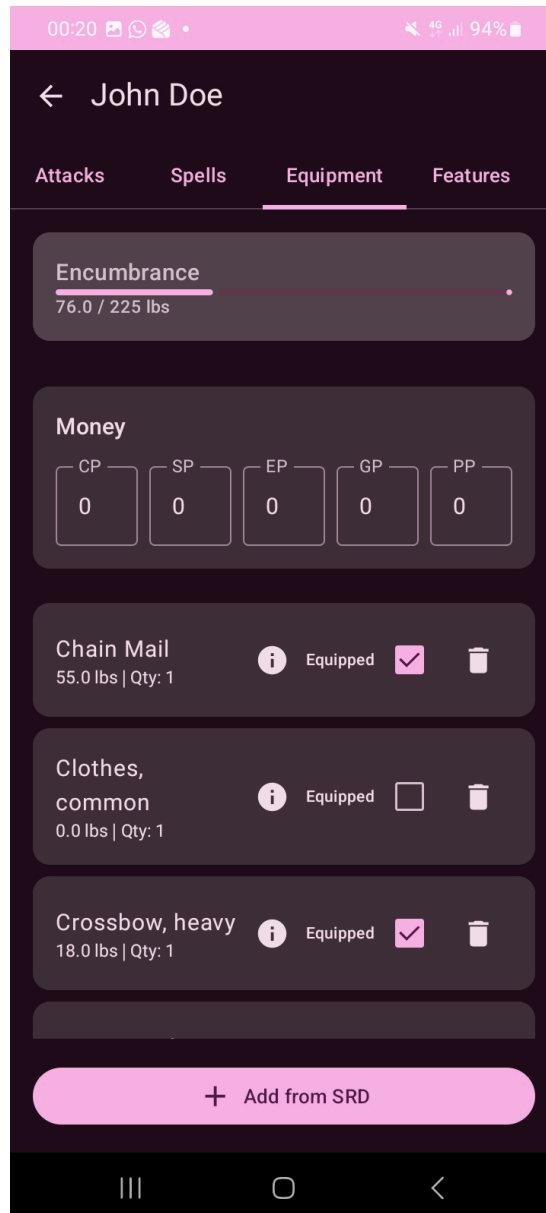


Figura 11 — Separador Equipment

Por fim, o separador **Features** apresenta características associadas à personagem, como traços raciais, características de classe, subclasse ou outras fontes. Ao selecionar uma característica, o utilizador pode consultar uma descrição mais detalhada, facilitando a utilização destas regras durante uma sessão de jogo.

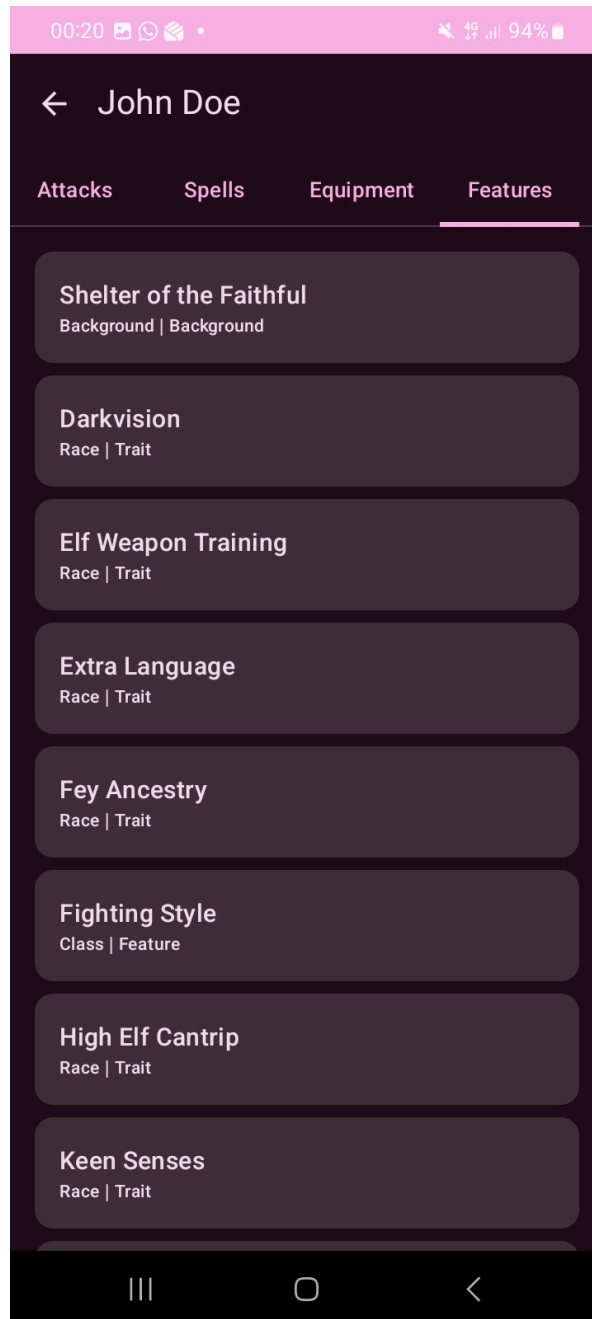


Figura 12 — Separador Features

Navegador de conteúdo SRD

Para além da gestão de personagens, a aplicação inclui um navegador de conteúdo SRD. Esta funcionalidade permite consultar informação de referência sobre diferentes elementos de Dungeons & Dragons 5e diretamente dentro da aplicação.

O utilizador pode navegar por categorias como **Classes**, **Races**, **Spells**, **Feats**, **Backgrounds** e **Equipment**. Ao selecionar uma categoria, é apresentada uma lista de elementos disponíveis. Quando o utilizador escolhe um item, a aplicação mostra os seus detalhes e, quando aplicável, também apresenta elementos relacionados, como características, variantes, subclasses, subraças ou equipamento inicial.

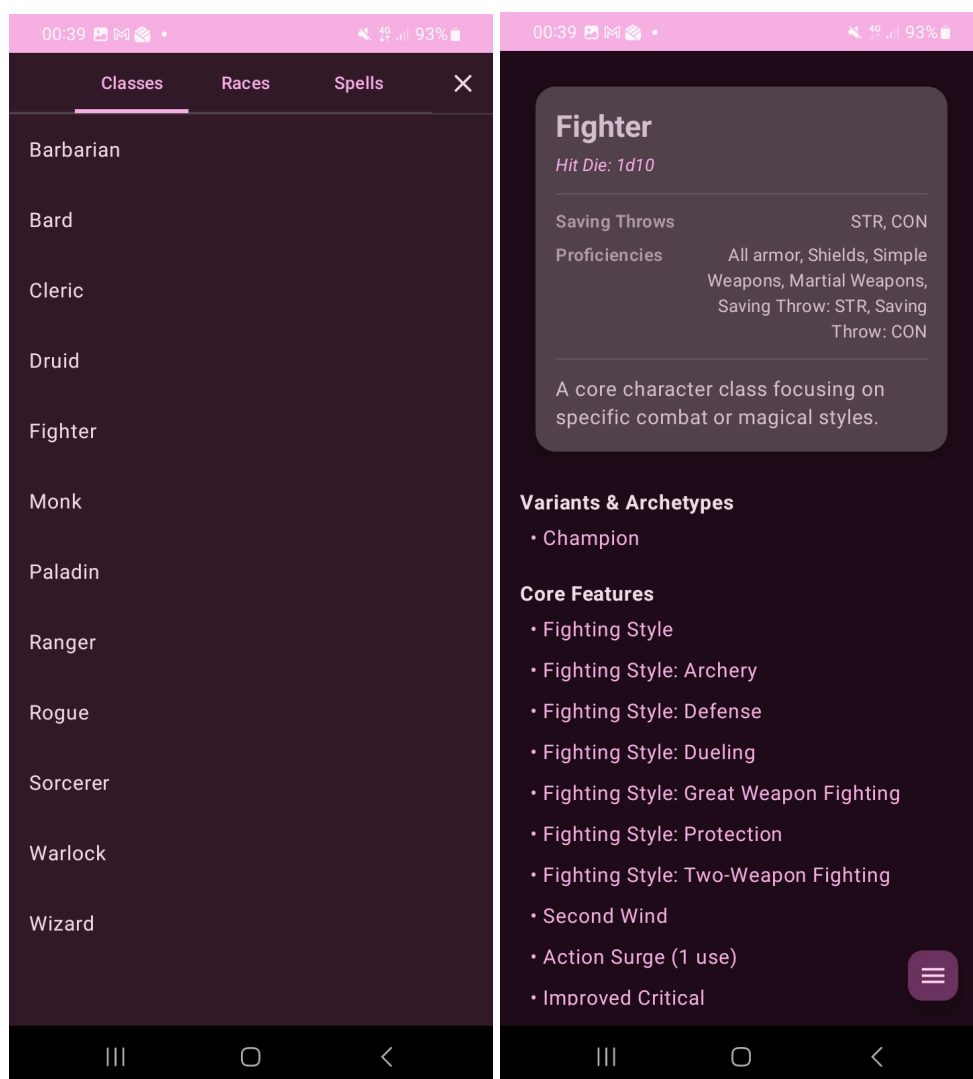


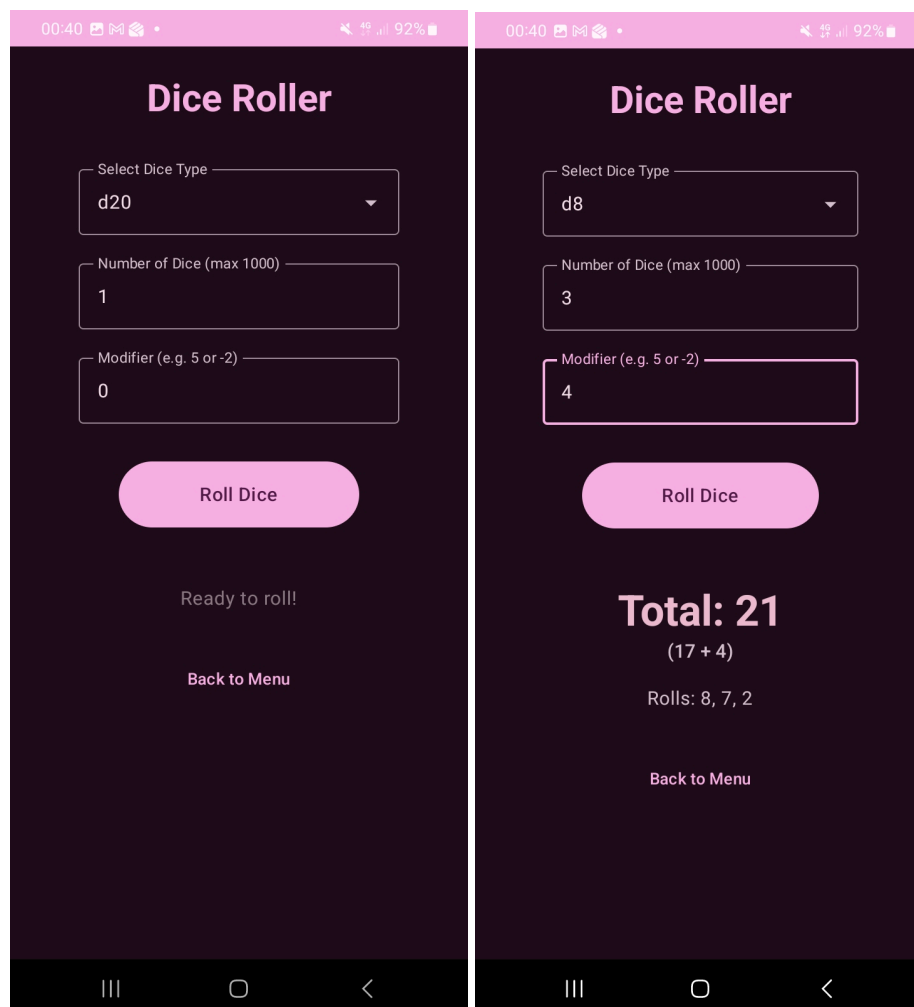
Figura 13 e 14 — Navegador SRD com lista de categorias e Detalhe de um item SRD selecionado

O navegador foi concebido para se adaptar à orientação do dispositivo. Em modo vertical, a navegação é feita através de um menu lateral que pode ser aberto e fechado. Em modo horizontal, a aplicação apresenta a lista de elementos e o conteúdo detalhado lado a lado, aproveitando melhor o espaço disponível no ecrã.

Simulador de lançamento de dados

A aplicação inclui também um ecrã de lançamento de dados, útil durante sessões de jogo. Nesta área, o utilizador pode escolher o tipo de dado a lançar, como d4, d6, d8, d10, d12, d20 ou d100. Também é possível indicar o número de dados e adicionar um modificador positivo ou negativo ao resultado.

Depois de carregar no botão **Roll Dice**, a aplicação apresenta o total obtido, o valor dos dados antes do modificador e a lista de resultados individuais. Esta funcionalidade permite simular rapidamente jogadas comuns de Dungeons & Dragons, como testes de perícia, ataques, dano ou verificações de resistência.



Figuras 15 e 16 — Ecrã Dice Roller antes do lançamento e Resultado de um lançamento de dados

Navegação e experiência de utilização

A navegação da aplicação foi pensada para ser simples e intuitiva. O utilizador começa no menu principal e, a partir daí, pode aceder às funcionalidades principais. Nos ecrãs secundários, existem botões de voltar, menus organizados e separadores que dividem a informação por contexto.

A utilização de cartões, botões de ação, separadores e formulários por etapas contribui para uma experiência mais organizada. Esta abordagem é especialmente importante numa aplicação deste tipo, pois uma ficha de personagem de **Dungeons & Dragons** contém muita informação. Ao dividir os dados por áreas, o Forge 5e facilita a consulta e evita que o utilizador tenha de procurar informação em ecrãs demasiado longos ou confusos.

De forma geral, o funcionamento da aplicação permite ao utilizador criar personagens, consultar a ficha completa, gerir equipamento e magias, visualizar características importantes, consultar conteúdo SRD e realizar lançamentos de dados. Assim, o Forge 5e funciona como uma ferramenta de apoio ao jogador, centralizando várias necessidades comuns de uma sessão de **Dungeons & Dragons** numa única aplicação Android.

Modelo de Dados

O modelo de dados da aplicação **Forge 5e** foi concebido para representar uma ficha de personagem de **Dungeons & Dragons 5e** de forma estruturada, persistente e extensível. Como a aplicação permite criar personagens, consultar os seus detalhes, gerir equipamento, magias, características e recursos, foi necessário separar a informação em várias entidades, evitando concentrar todos os dados numa única estrutura.

A aplicação utiliza uma base de dados local com **Room**, definida na classe `AppDatabase`. Esta base de dados contém as entidades principais relacionadas com as personagens: `CharacterEntity`, `CharacterEquipmentEntity`, `CharacterSpellEntity`, `CharacterTrackerEntity` e `CharacterFeatureEntity`. A base de dados é guardada localmente com o nome `forge5e_database`, permitindo que as personagens criadas pelo utilizador permaneçam disponíveis mesmo após fechar a aplicação.

Entidade CharacterEntity

A entidade principal do modelo de dados é a `CharacterEntity`, que representa uma personagem criada pelo utilizador. Esta entidade corresponde à tabela `characters` e possui um identificador único gerado automaticamente.

Esta entidade guarda os dados base da personagem, como nome, classe, subclasse, raça, subraça, background e nível. Para além dos nomes apresentados ao utilizador, são também guardados índices como `classIndex`, `raceIndex`, `subclassIndex` e `backgroundIndex`, que permitem relacionar a personagem com os dados obtidos através da API SRD 5e.

A personagem contém também os seis atributos principais do sistema de Dungeons & Dragons: Força, Destreza, Constituição, Inteligência, Sabedoria e Carisma. Estes valores são usados para calcular modificadores, perícias, testes de resistência, iniciativa e outros elementos da ficha.

Além dos atributos, a entidade guarda informação relativa a pontos de vida, dados de vida, testes contra morte, nível de exaustão, experiência, proficiências, idiomas, moedas, magias, bônus adicionais e regras opcionais escolhidas durante a criação da personagem.

Desta forma, a `CharacterEntity` funciona como o núcleo da ficha digital da personagem.

Entidades associadas à personagem

Para manter o modelo de dados organizado, a aplicação utiliza entidades complementares associadas à personagem através do campo `characterId`.

Entidade	Finalidade
<code>CharacterEquipmentEntity</code>	Guarda os itens de equipamento associados à personagem.
<code>CharacterSpellEntity</code>	Guarda as magias conhecidas ou preparadas pela personagem.
<code>CharacterFeatureEntity</code>	Guarda características provenientes da raça, classe, subclasse ou background.
<code>CharacterTrackerEntity</code>	Guarda recursos personalizados, como utilizações de habilidades ou recursos que recuperam em descanso curto ou longo.

A entidade `CharacterEquipmentEntity` representa os itens pertencentes à personagem. Cada item possui informação como índice do equipamento, nome, quantidade, estado de equipamento, peso, atributo selecionado e uso a duas mãos. A chave primária é composta por `characterId` e `equipmentIndex`, garantindo que cada personagem pode ter o seu próprio conjunto de itens.

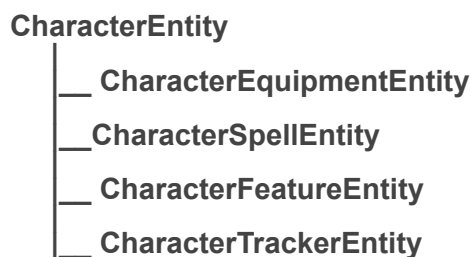
A entidade `CharacterSpellEntity` representa as magias associadas à personagem. Para além do índice e nome da magia, guarda informação sobre usos atuais, usos máximos, tipo de recuperação e se a magia está preparada. Isto permite diferenciar personagens que conhecem magias de personagens que precisam de preparar magias.

A entidade `CharacterFeatureEntity` guarda características da personagem, como traços raciais, características de classe ou características provenientes do background. Esta entidade inclui o nome, a origem e o tipo da característica, permitindo apresentar a informação de forma organizada no separador **Features** da ficha.

Por fim, a entidade `CharacterTrackerEntity` permite guardar recursos associados à personagem, como habilidades com utilizações limitadas. Cada recurso possui um valor atual, um valor máximo e um tipo de recuperação, por exemplo descanso curto ou descanso longo.

Relação entre entidades

O modelo de dados pode ser representado de forma simplificada da seguinte forma:



A entidade `CharacterEntity` representa a personagem principal. As restantes entidades representam informação dependente dessa personagem. Esta estrutura permite que cada personagem tenha o seu próprio inventário, lista de magias, características e recursos, sem misturar dados entre personagens diferentes.

Embora as relações não estejam formalmente representadas com objetos relacionais complexos do Room, a associação é feita através do campo `characterId`, que funciona como referência ao identificador da personagem.

Acesso aos dados locais

O acesso aos dados persistidos é feito através do `CharacterDao`. Este DAO contém operações para consultar, inserir, atualizar e eliminar personagens, equipamento, magias, recursos e características. Algumas consultas devolvem `Flow`, permitindo que a interface seja atualizada automaticamente quando os dados da base de dados são alterados.

O `CharacterDao` possui também métodos específicos para cada entidade associada. Por exemplo, existem métodos para obter o equipamento de uma personagem, obter as suas magias, consultar os seus recursos e sincronizar as características associadas.

Esta abordagem é adequada à arquitetura MVVM utilizada no projeto, pois permite que os repositórios e ViewModels observem alterações na base de dados e atualizem a interface de forma reativa.

Integração com a API SRD 5e

Para além dos dados guardados localmente, a aplicação utiliza a **D&D 5e SRD API** como fonte externa de informação. Esta API fornece dados de referência sobre elementos do sistema de jogo, como classes, raças, magias, equipamento, talentos, backgrounds, proficiências, subclasses, subraças, características de classe e traços raciais.

A comunicação com a API é feita através de **Retrofit**, configurado no objeto **NetworkClient**. O URL base utilizado é <https://www.dnd5eapi.co/>, e a configuração de JSON foi preparada para ignorar campos desconhecidos e aceitar valores convertidos, tornando a aplicação mais resistente a pequenas alterações na resposta da API.

Os endpoints disponíveis estão definidos na interface **SrdApiService**. Esta interface inclui métodos para obter listas gerais, como classes, raças, magias, equipamento e talentos, bem como métodos para consultar detalhes específicos através de um **index**.

A API também é utilizada para obter informação contextual, como subclasses de uma classe, características de uma classe, níveis de classe, magias disponíveis para uma classe, subraças de uma raça, traços raciais e equipamento inicial.

O acesso à API é abstraído pelo **SrdRepository**, que encapsula as chamadas remotas e devolve os resultados aos ViewModels. Este repositório utiliza **Result** para representar o sucesso ou falha das operações, evitando que a interface tenha de lidar diretamente com os detalhes da comunicação HTTP.

Dados locais vs. dados remotos

O modelo de dados da aplicação combina dois tipos de informação:

Tipo de dados	Origem	Exemplos
Dados locais	Base de dados Room	Personagens criadas, equipamento da personagem, magias escolhidas, recursos e características associadas.
Dados remotos	API SRD 5e	Classes, raças, magias, equipamento, backgrounds, talentos, proficiências e características oficiais do SRD.

Esta separação permite que a aplicação guarde localmente apenas os dados específicos do utilizador, enquanto os dados de referência são obtidos da API quando necessário. Por exemplo, uma personagem guarda o índice da sua classe ou raça, e esse índice pode ser usado para consultar informação detalhada na API SRD 5e.

Assim, o modelo de dados torna-se mais leve e flexível, evitando duplicar localmente toda a informação do sistema de Dungeons & Dragons.

Conclusão

O desenvolvimento da aplicação **Forge 5e** permitiu concretizar a proposta apresentada inicialmente: uma aplicação Android para gestão de personagens de **Dungeons & Dragons 5e**. Ao longo do projeto, a ideia inicial evoluiu para uma solução mais completa, incluindo criação de personagens, consulta de fichas detalhadas, gestão de equipamento, magias, características, recursos, navegador de conteúdo SRD e simulador de lançamento de dados.

A aplicação cumpre o objetivo principal de funcionar como uma ficha digital interativa, permitindo ao utilizador criar e consultar personagens de forma organizada. A divisão da ficha por separadores facilita a navegação e torna a utilização mais prática durante uma sessão de jogo, onde o acesso rápido a atributos, pontos de vida, ataques, magias e equipamento é essencial.

Do ponto de vista técnico, o projeto permitiu aplicar conceitos importantes de desenvolvimento Android. A utilização da arquitetura **MVVM** contribuiu para uma melhor separação de responsabilidades entre interface, lógica da aplicação e acesso aos dados. A interface foi construída com **Jetpack Compose**, permitindo criar ecrãs reativos e organizados. A persistência local foi implementada com **Room**, garantindo que as personagens criadas permanecem guardadas no dispositivo.

Outro ponto relevante foi a integração com a **SRD 5e API**, que permitiu enriquecer a aplicação com dados reais do sistema de jogo. Esta integração tornou possível consultar classes, raças, magias, equipamento, talentos, backgrounds e outras informações diretamente na aplicação, reduzindo a necessidade de inserir manualmente todos esses dados.

O projeto apresenta também uma complexidade superior a uma aplicação CRUD simples. Para além de criar, listar e eliminar personagens, o Forge 5e inclui lógica relacionada com regras de Dungeons & Dragons, como atributos, pontos de vida, proficiências, magias, equipamento, carga, recursos, descansos e lançamento de dados. Esta complexidade demonstra a aplicação prática dos conhecimentos adquiridos ao longo da unidade curricular.

Apesar de funcional, a aplicação pode ainda ser melhorada no futuro. Algumas possibilidades de evolução incluem a exportação da ficha de personagem para PDF, sincronização na cloud, criação de campanhas, partilha de personagens entre utilizadores, edição mais avançada da ficha, suporte a conteúdos personalizados e maior expansão das regras opcionais.

Em conclusão, o **Forge 5e** cumpre os objetivos definidos para o projeto final, apresentando uma aplicação Android funcional, temática e tecnicamente estruturada. O projeto permitiu consolidar conhecimentos de desenvolvimento mobile, arquitetura MVVM, persistência local, consumo de APIs externas e construção de interfaces modernas, resultando numa aplicação útil para jogadores de Dungeons & Dragons 5e.